

AgriSense AI

Intelligent, Multilingual Farm Decision Companion



Technical Documentation and System Architecture

Platform:	AgriSense AI
Architecture:	Full Stack, API-First, Modular AI
Backend:	Python 3.11, FastAPI
Frontend:	React 19, Vite
AI Stack:	PyTorch, Hugging Face, LLMs
Primary Dataset:	PlantVillage
Documentation:	Technical Project Report

Abstract

AgriSense AI is a full-stack, AI-powered farm companion designed to help smallholder farmers diagnose crop diseases, understand field health, make market decisions, and access agricultural intelligence through an intuitive multilingual interface.

The platform combines computer vision, large language models, field-level analytics, market trend analysis, weather-aware recommendations, multilingual interaction, voice input, community question answering, and outbreak intelligence into a single platform.

Its API-first architecture allows the same intelligence to be consumed through the web application and extended to future interfaces such as WhatsApp. The system is designed around a central idea: agricultural AI should not merely identify problems, but should help farmers understand the situation and decide what to do next.

Keywords: Agriculture, Artificial Intelligence, Computer Vision, Plant Disease Detection, Hugging Face, Transformers, FastAPI, React, LLM, Groq, Multilingual AI, Field Analytics, Decision Support.

Contents

1	Introduction	3
1.1	Core Objectives	3
2	System Overview	3
2.1	High-Level Architecture	3
3	Technology Stack	4
4	Crop Disease Detection	4
4.1	Model	4
4.2	PlantVillage Dataset	4
4.3	Inference Pipeline	5
5	Multi-Image Field Scan	5
5.1	Example Field Scan Result	5
6	Market Intelligence	6
6.1	Regression Models	6
6.2	Market Intelligence Pipeline	6
7	AI Sell/Hold Advisor	7
8	Weather Intelligence	7
9	Multilingual and Voice Interaction	7
9.1	Voice Architecture	7
10	Community Intelligence	8
11	Outbreak Intelligence	8
12	API Architecture	8
13	Request Flow	9
14	Future WhatsApp Farm Companion	9
14.1	Proposed Architecture	10
14.2	Example Interaction	10
15	Platform Capability Overview	10
16	End-to-End Decision Pipeline	11
17	Configuration and Security Considerations	11
18	Current Limitations	12
19	Future Work	12
20	Conclusion	13
A	Local Development	13
B	Representative API Structure	13

1 Introduction

Agriculture involves continuous decision-making under uncertainty. Farmers must identify crop diseases, understand the health of their fields, monitor weather conditions, evaluate market prices, and determine the appropriate time to sell their produce.

These tasks are often fragmented across different information sources. AgriSense AI addresses this fragmentation by creating a unified AI-powered farm companion.

The platform transforms a simple crop image, a farmer's question, or a market query into an actionable agricultural intelligence workflow.

1.1 Core Objectives

The primary objectives of AgriSense AI are:

- Detect potential crop diseases from plant images.
- Analyze multiple plants to estimate field-level health.
- Provide indicative potential yield-loss estimates.
- Analyze crop market trends.
- Provide weather-aware Sell/Hold recommendations.
- Support multilingual agricultural interaction.
- Enable voice-based interaction in regional languages.
- Provide community-driven agricultural question answering.
- Identify potential disease clusters using spatial clustering.
- Provide an API-first foundation for future WhatsApp integration.

2 System Overview

AgriSense AI follows a modular full-stack architecture in which the React frontend communicates with a Python FastAPI backend.

The backend orchestrates specialized components for computer vision, LLM reasoning, market intelligence, weather services, translation, field analytics, and community intelligence.

2.1 High-Level Architecture

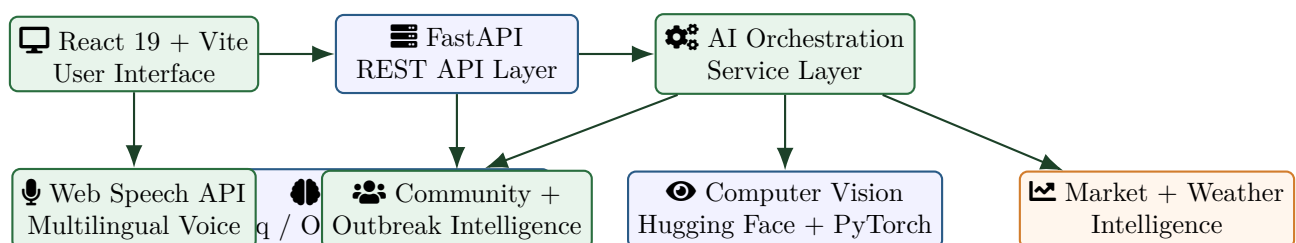


Figure 1: High-level AgriSense AI architecture

3 Technology Stack

Layer	Technologies
Frontend	React 19, Vite, JavaScript, Axios, React Router, Recharts
Backend	Python 3.11, FastAPI, Pydantic, Uvicorn, REST APIs
Deep Learning	PyTorch, Hugging Face Transformers, MobileNetV2
Dataset	PlantVillage, 38 plant disease and healthy-state classes
Computer Vision	Image classification pipeline, Pillow, Hugging Face Transformers
LLM Layer	Groq, OpenAI, Anthropic compatible providers, tool/function calling
Machine Learning	Scikit-learn, Linear Regression, Polynomial Regression, TF-IDF, cosine similarity, DBSCAN
Numerical/Data Processing	NumPy, Pandas
Weather	OpenWeather API or mock weather provider
Voice	Browser Web Speech API, SpeechRecognition / webkitSpeechRecognition
Visualization	Recharts, geographic outbreak visualization
API Documentation	FastAPI Swagger UI / OpenAPI 3.1
Future Interface	WhatsApp Business / webhook integration architecture

Table 1: AgriSense AI technology stack

4 Crop Disease Detection

4.1 Model

AgriSense AI uses the Hugging Face Transformers ecosystem with a pretrained MobileNetV2 image classification model:

`linkanjarad/mobilenet_v2_1.0_224-plant-disease-identification`

The model is fine-tuned on the PlantVillage dataset.

4.2 PlantVillage Dataset

The PlantVillage dataset provides labeled plant leaf images covering 38 disease and healthy-state classes across multiple crops.

Examples include:

- Tomato
- Potato
- Corn
- Grape
- Apple
- Pepper

4.3 Inference Pipeline



Figure 2: Crop disease inference pipeline

The first inference may download several hundred megabytes of model weights. The model is subsequently cached locally for later inference.

5 Multi-Image Field Scan

A single leaf may not accurately represent the condition of an entire field. AgriSense therefore supports multi-image field scanning.

The user can provide multiple images representing different plants or locations within a field.

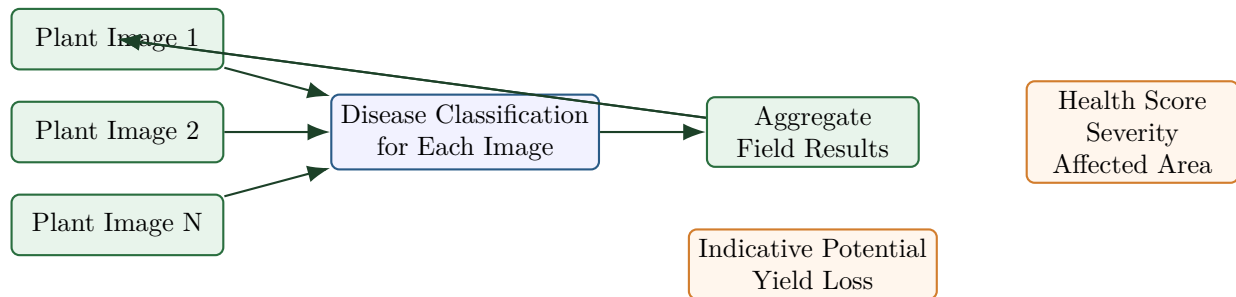


Figure 3: Field Scan workflow

The system aggregates individual predictions to generate:

- Field health information
- Condition breakdown
- Number of affected plants
- Average severity
- Estimated affected field area
- Indicative potential yield-loss estimate

5.1 Example Field Scan Result

A representative scan may produce:

Metric	Result
Plants scanned	2
Plants showing symptoms	2/2
Cherry with Powdery Mildew	50%
Tomato with Early Blight	50%
Average severity	49%
Estimated affected field area	30%
Indicative potential yield loss	12.5%

Table 2: Example field-scan output

The yield-loss value is intended as a decision-support estimate rather than a scientifically validated agronomic forecast.

6 Market Intelligence

AgriSense contains a market intelligence module based on Python, NumPy, Pandas, and Scikit-learn.

The current prototype supports crops including:

Tomato Potato Onion Wheat Grapes

The prototype can generate simulated historical commodity-price data and apply regression models for trend analysis and short-term forecasting.

6.1 Regression Models

The system supports:

- Linear Regression
- Polynomial Regression

For polynomial regression, the general form is:

$$y = \beta_0 + \beta_1x + \beta_2x^2 + \dots + \beta_nx^n$$

where x represents the temporal input and y represents the estimated crop price.

6.2 Market Intelligence Pipeline

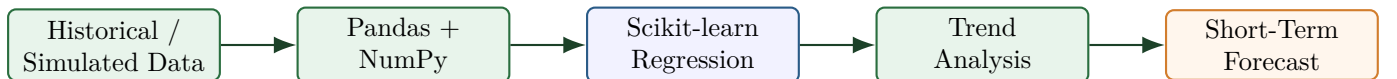


Figure 4: Market intelligence pipeline

The architecture allows the simulated data source to be replaced by real agricultural market sources such as Agmarknet or data.gov.in in a production deployment.

7 AI Sell/Hold Advisor

The Sell/Hold Advisor represents the decision-support layer of AgriSense.

Instead of relying on a single data point, the advisor can combine multiple contextual signals.

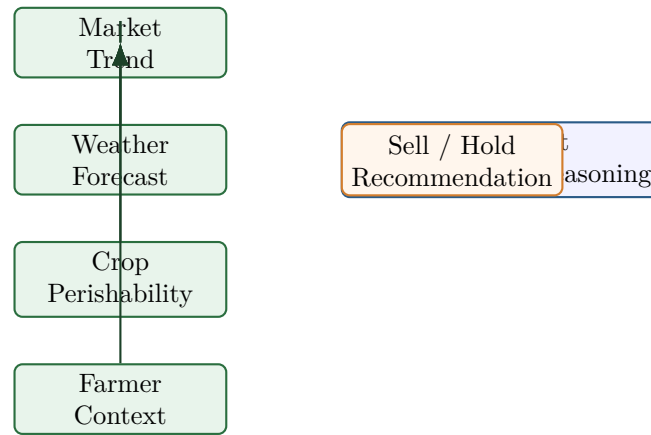


Figure 5: AI Sell/Hold decision architecture

The LLM provider can be configured to use Groq, OpenAI, Anthropic, or a demo mode.

The system supports tool/function calling so that the LLM can retrieve relevant information before producing its recommendation.

8 Weather Intelligence

The weather service supports two modes:

1. OpenWeather API
2. Mock weather provider

The weather layer is intentionally implemented as a replaceable service. This allows real weather information to be incorporated into decision-making without changing the overall application architecture.

9 Multilingual and Voice Interaction

AgriSense supports 10 languages:

English	Hindi	Kannada	Tamil
Telugu	Marathi	Bengali	Gujarati
Punjabi	Malayalam		

The system can use its LLM layer for translation and multilingual response generation.

9.1 Voice Architecture

The current voice interface uses browser-native speech recognition through:

SpeechRecognition / webkitSpeechRecognition

The architecture is:

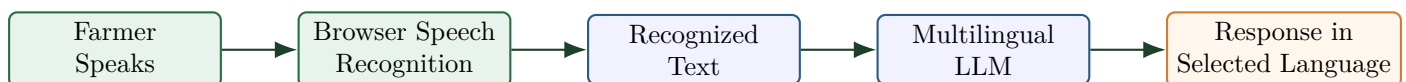


Figure 6: Multilingual voice interaction pipeline

The current implementation does not use Vapi or Bhashini. The architecture can, however, be extended to support specialized speech and Indian-language APIs in future versions.

10 Community Intelligence

The community layer allows farmers to ask agricultural questions and receive contextual responses.

To identify related questions, AgriSense uses:

- TF-IDF vectorization
- Cosine similarity
- LLM-based response generation

For two vectors A and B , cosine similarity is:

$$\text{cosine}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

This allows the system to retrieve semantically related community questions and use them as contextual information.

11 Outbreak Intelligence

AgriSense contains an outbreak intelligence module based on spatial clustering.

The prototype uses disease reports containing geographical coordinates and applies the DBSCAN clustering algorithm.

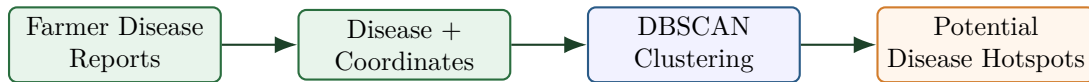


Figure 7: Outbreak intelligence workflow

In production, simulated reports can be replaced with real, consent-based farmer observations stored in a persistent database.

12 API Architecture

The FastAPI backend exposes modular REST endpoints.

Endpoint	Purpose
/api/diagnose	Crop disease diagnosis from an image
/api/field-scan	Multi-image field analysis
/api/market/crops	List supported crops
/api/market/trend/{crop}	Market trend analysis
/api/advisor/sell-hold	AI Sell/Hold recommendation
/api/chat	Conversational agricultural AI
/api/outbreak/map	Outbreak cluster visualization
/api/translate/languages	Supported languages
/api/translate	Translation and multilingual generation
/api/qa	Community question listing
/api/qa/ask	Community Q&A interaction
/api/health	Backend health monitoring

Table 3: AgriSense AI REST API

Interactive OpenAPI documentation is automatically generated by FastAPI and can be accessed during local development at:

`http://localhost:8000/docs`

13 Request Flow

A typical crop diagnosis request follows the following architecture:

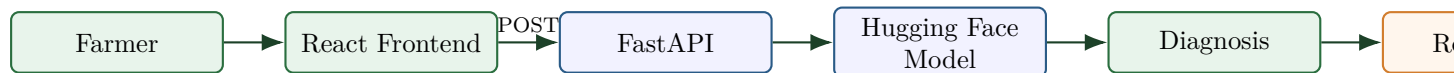


Figure 8: Crop diagnosis API request flow

14 Future WhatsApp Farm Companion

A key future extension is the integration of AgriSense with WhatsApp.

The existing API-first design means that the web application does not need to be the only interface through which users access the intelligence layer.

14.1 Proposed Architecture

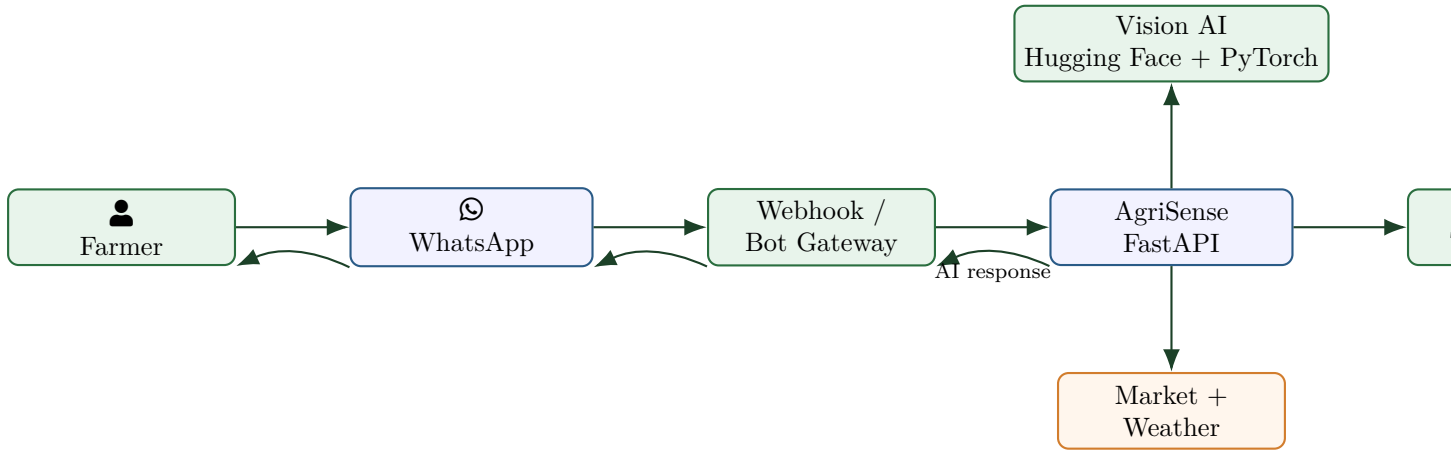


Figure 9: Proposed WhatsApp Farm Companion architecture

A farmer could send a crop image directly through WhatsApp. The bot would forward the image to the appropriate AgriSense endpoint and return the diagnosis.

Similarly, natural-language queries can be routed to the chat endpoint, market questions can access market APIs, and Sell/Hold requests can invoke the AI advisor.

14.2 Example Interaction

Farmer:

“ . ?”

WhatsApp Bot:

Receives message → calls `/api/chat` → multilingual AI processing → returns Kannada response.

The current project contains a WhatsApp-oriented interface concept rather than a production WhatsApp Business integration. The API architecture is designed to make such an integration possible without rebuilding the core AI services.

15 Platform Capability Overview

The following chart summarizes the major intelligence modules currently represented in the platform.

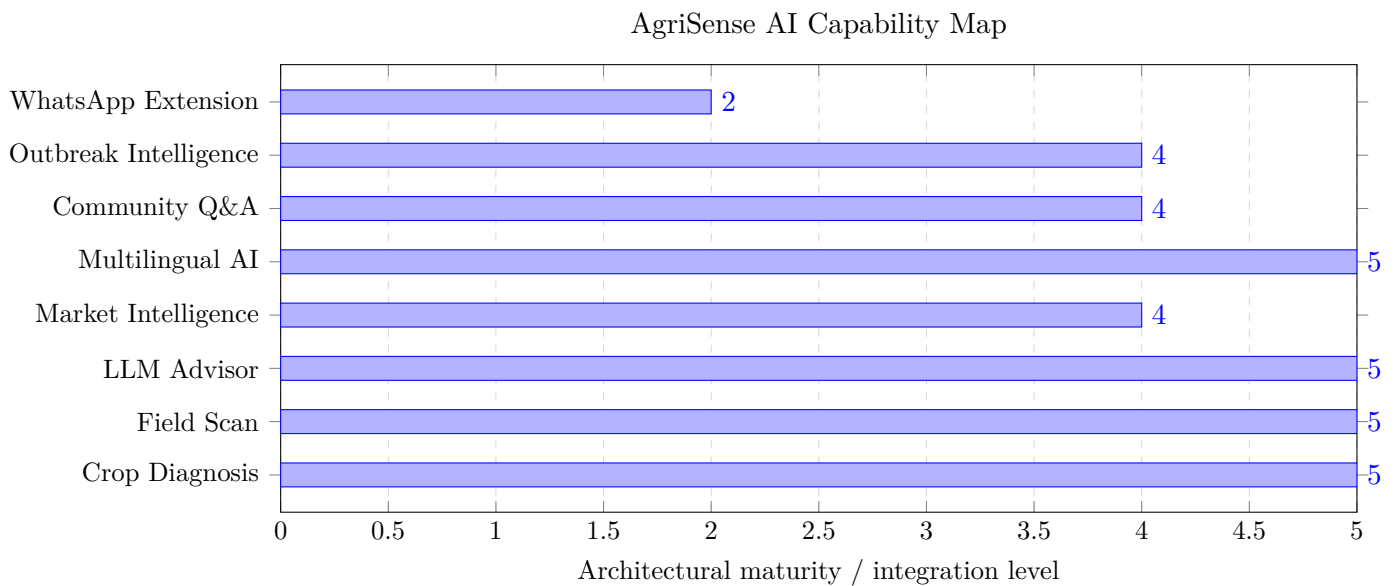


Figure 10: Relative implementation and integration status of major capabilities

The chart is intended as a qualitative architectural overview rather than a benchmark.

16 End-to-End Decision Pipeline

The core value proposition of AgriSense can be represented as a progressive decision pipeline:

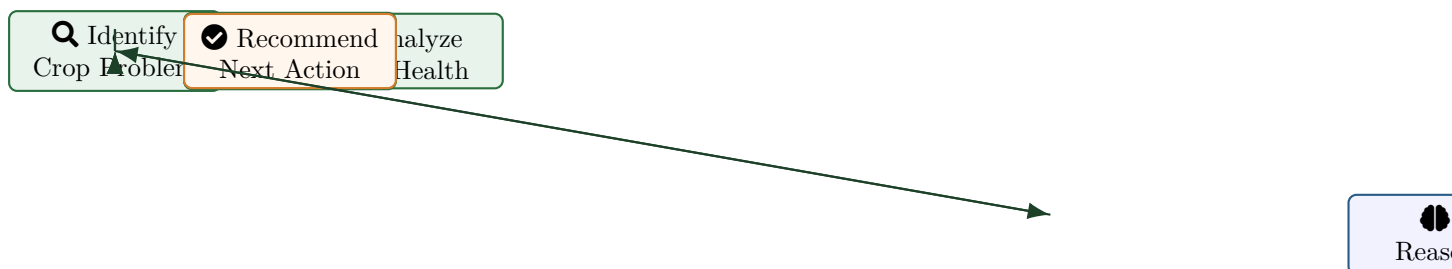


Figure 11: AgriSense decision-support pipeline

This progression represents the central philosophy of the platform:

What is happening? → How widespread is it? → What could it mean? →
What should I do?

17 Configuration and Security Considerations

API credentials are stored through environment variables rather than being embedded into the frontend.

Example configuration:

```

1 LLM_PROVIDER=groq
2 GROQ_API_KEY=gsk_...
3 WEATHER_PROVIDER=openweather
4 OPENWEATHER_API_KEY=...

```

The backend can also operate in mock/demo modes when external credentials are unavailable.

For production deployment, recommended improvements include:

- Persistent database storage.
- Authentication and authorization.
- HTTPS.
- Rate limiting.
- Secure secret management.
- Farmer-consent mechanisms for community data.
- Validation and monitoring of external API responses.
- Model confidence thresholds and human agronomist escalation.

18 Current Limitations

The current prototype has several limitations that should be addressed before production deployment.

1. The PlantVillage-trained model may perform differently on real-world field photographs because PlantVillage images are often cleaner and more controlled than field conditions.
2. Yield-loss calculations are indicative estimates and should not be interpreted as validated agronomic predictions.
3. Market data may be simulated in prototype mode.
4. Weather may use mock data when an OpenWeather API key is unavailable.
5. Outbreak intelligence currently uses simulated community observations.
6. The WhatsApp integration is currently an architectural extension rather than a production WhatsApp Business deployment.
7. Browser-based speech recognition varies across browsers and platforms.

These limitations provide clear directions for future development and real-world validation.

19 Future Work

The next stage of AgriSense can include:

- Integration with real agricultural market datasets.
- Integration with real-time government and agricultural APIs.
- Persistent farmer profiles and field histories.
- Real WhatsApp Business API integration.
- Dedicated multilingual speech-to-text and text-to-speech systems.
- Integration with Bhashini or specialized Indian-language speech services.
- Vapi-based voice agents where appropriate.
- Fine-tuning or domain adaptation using region-specific crop imagery.
- Weather-driven disease-risk prediction.
- Satellite and remote-sensing based field monitoring.
- IoT sensor integration for soil moisture and environmental conditions.
- Agronomist-in-the-loop verification for high-risk diagnoses.

- Personalized recommendations based on farmer history and local conditions.

20 Conclusion

AgriSense AI is designed as an API-first agricultural intelligence platform rather than simply a web application.

Its architecture combines computer vision, machine learning, large language models, market intelligence, weather information, multilingual interaction, voice input, community knowledge, and spatial clustering into a unified farm companion.

The platform transforms the farmer's journey from identifying a problem to understanding its scale, estimating its potential impact, evaluating external conditions, and deciding what to do next.

The modular backend allows the same intelligence to be accessed through multiple interfaces. The current web application demonstrates the platform, while the API architecture provides a foundation for future channels such as WhatsApp.

The long-term vision is to make advanced agricultural intelligence accessible, understandable, multilingual, and actionable for farmers.

AgriSense AI

From crop diagnosis to better decisions.

A Local Development

Backend:

```
1 cd backend
2 py -3.11 -m venv venv
3 venv\Scripts\activate
4 pip install -r requirements.txt
5 copy .env.example .env
6
7 python -m uvicorn app.main:app --reload --port 8000
```

Frontend:

```
1 cd frontend
2 npm install
3 copy .env.example .env
4 npm run dev
```

Backend API documentation:

<http://localhost:8000/docs>

Frontend:

<http://localhost:5173>

B Representative API Structure

```
1 POST /api/diagnose
2 POST /api/field-scan
3
4 GET /api/market/crops
5 GET /api/market/trend/{crop}
```

```
6
7 POST /api/advisor/sell-hold
8 POST /api/chat
9
10 GET /api/outbreak/map
11
12 GET /api/translate/languages
13 POST /api/translate
14
15 GET /api/qa
16 POST /api/qa/ask
17
18 GET /api/health
```